

第 01 章·Numpy 及其基本使用

目录

Numpy 及其基本使用

01 数组基础：创建、属性、索引、广播与随机数

02 数组运算：向量、矩阵、逻辑筛选与统计

03 线性代数：逆、方程组、分解与 SVD

04 多项式与拟合：poly1d、求根与最小二乘拟合

05 综合案例：SVD 图像压缩与实验数据拟合

06 常见误区与技巧

1

1

7

10

13

16

19

Numpy 及其基本使用

01 数组基础：创建、属性、索引、广播与随机数

本节对应原版 1.1 的内容，并增补学习目标、常见误区、思考题与延伸阅读。配套图：images/reshape.png、images/broadcast.png (由 build/make_chap1_figures.py 生成)。

本节目标

- 理解 ndarray 是什么，以及它与 Python 列表的区别；
- 会创建常见数组 (array、zeros、ones、full、arange、linspace、diag、eye)；
- 掌握 shape、ndim、dtype、size、itemsize 等属性；
- 掌握索引、切片、花式索引、布尔掩码；
- 理解视图 (view) 与拷贝 (copy)，知道 reshape ravel flatten copy 的区别；
- 掌握广播规则，能手工判断形状是否兼容；
- 使用新版随机数生成器 Generator (default_rng) 生成可复现随机数。

先修

- Python 列表、元组、循环、函数；了解变量类型的含义。

1.1 为什么用 NumPy

Python 列表方便但“慢”，且不能直接做向量化运算。NumPy 的 ndarray 由 C 实现，元素类型统一，支持逐元素运算与广播，这正是科学计算需要的能力。

特性	Python list	NumPy ndarray
元素类型	可混合	必须统一 (dtype)
逐元素运算	需循环/列表推导	直接 <code>a + b</code>
内存	对象指针数组	连续内存块

特性	Python list	NumPy ndarray
速度	慢	快 (向量化)

1.2 创建数组

1.2.1 从列表/元组创建

```
import numpy as np
```

```
x = np.array([1, 2, 3, 4, 5])          # 一维数组
y = np.array([[1, 2, 3], [4, 5, 6]])  # 二维数组 (矩阵)
print(x, x.shape, x.dtype)
print(y, y.shape, y.ndim)
print(type(x))

[1 2 3 4 5] (5,) int64
[[1 2 3]
 [4 5 6]] (2, 3) 2
<class 'numpy.ndarray'>
```

注意：子数组长度必须一致；`np.array([[1,2],[3]])` 会报错或产生 object 数组。

1.2.2 指定数据类型

```
x = np.array([1, 2, 3], dtype=np.int32)
print(x.dtype, x.itemsize)  # int32 4

x2 = x.astype(np.float64)
print(x2.dtype, x2.itemsize) # float64 8

## 与 Python 列表互转
lst = x.tolist()           # 或 list(x)
arr = np.array(lst)
```

内存开销：`astype` 默认返回新数组 (拷贝)；如仅查看数值但不改，可避免不必要的转换。

1.2.3 快速创建

```
np.zeros((2, 3))          # 全 0, 形状 (2,3)
np.ones((2, 3))           # 全 1
np.full((2, 3), 42)       # 全 42
np.arange(0, 11, 2)       # [ 0  2  4  6  8 10]
np.linspace(0, 10, 5)     # [ 0.  2.5  5.  7.5 10. ]
np.diag([1, 2, 3])        # 对角线矩阵
np.eye(3)                 # 3x3 单位矩阵
np.eye(3, 4)              # 3x4, 主对角线为 1
np.eye(3, k=1)            # 主对角线上方一条对角线为 1

print(np.arange(5))        # [0 1 2 3 4]
print(np.linspace(0, 1, 3)) # [0.  0.5 1. ]
print(np.diag([1, 2, 3]))

[0 1 2 3 4]
[0.  0.5 1. ]
[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

提醒: zeros/ones/full 的形状参数是**元组** (两个括号); arange 类似 range, 终点是开区间。

1.2.4 数组属性

```
a = np.arange(24).reshape(2, 3, 4)
print("shape:", a.shape)    # 各维长度
print("ndim :", a.ndim)     # 维数 3
print("size :", a.size)     # 元素总数 24
print("dtype:", a.dtype)    # 数据类型
print("itemsize:", a.itemsize) # 单元素字节数
```

1.3 变形、转置与拼接

1.3.1 reshape

```
arr = np.arange(6)
print(arr.reshape((2, 3))) # 或 arr.reshape(2, 3)
print(arr.reshape((3, -1))) # -1 自动推断: 3×2
print(arr.reshape(-1))     # 展平为一维
```

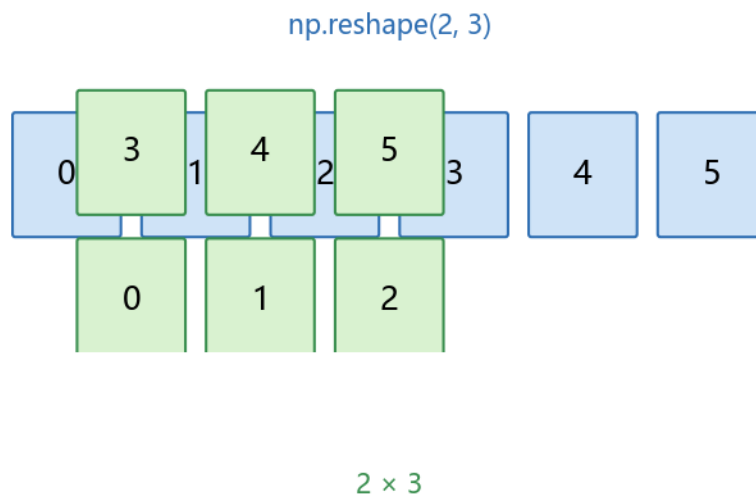


Figure 1: reshape 示意图

- 总元素数必须相等;
- reshape 在内存连续时通常返回**视图** (共享内存); 需要独立副本时用 `arr.reshape(...).copy()`。

1.3.2 展平: ravel vs flatten

```
a = np.array([[1, 2, 3], [4, 5, 6]])
a.ravel()    # 视图 (尽可能不拷贝)
a.flatten()  # 总是拷贝
a.reshape(-1) # 展平
```

1.3.3 转置

```
a = np.arange(6).reshape(2, 3)
a.T    # 转置 (3, 2)
np.transpose(a)    # 等价
np.transpose(a, axes=(1, 0)) # 高维时可指定轴的顺序
```

1.3.4 拼接与堆叠

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])
np.concatenate((a, b), axis=0)    # 沿 0 轴拼接: (3,2)
np.concatenate((a, a), axis=1)    # 沿 1 轴拼接: (2,4)
np.vstack((a, b))                 # 垂直堆叠
np.hstack((a, a))                 # 水平堆叠
np.stack((a, b), axis=0)          # 新增维度堆叠 (2,2)
np.column_stack((a, b))           # 一维数组按列堆叠 (2,2)
```

1.3.5 增删元素

```
a = np.array([1, 2, 3])
np.append(a, 4)                   # 返回新数组 [1 2 3 4]
np.delete(a, 1)                   # 返回新数组 [1 3]
a[[0, 2]]                         # 花式索引取第 0、2 个
a[a != 3]                         # 布尔掩码过滤
```

频繁 append/delete 会反复拷贝，性能差；大数据请先想好形状或用列表收集。

1.4 索引与切片

1.4.1 基本索引与切片 (视图)

```
a = np.arange(10)
a[0]          # 标量
a[2:5]        # 视图, 切片
a[::-1]       # 反转
```

二维数组:

```
m = np.arange(12).reshape(3, 4)
m[0, 1]       # 第 0 行第 1 列
m[:, 1]       # 第 1 列 (视图)
m[1:3, 1:3]   # 子矩阵 (视图)
```

1.4.2 花式索引 (fancy indexing, 拷贝)

```
m = np.arange(12).reshape(3, 4)
print(m[[0, 2], [1, 3]])    # 取 (0,1) 和 (2,3) → [1 11]
print(m[np.ix_([0, 2], [1, 3])]) # 行 × 列交叉 → 2×2

[ 1 11]
[[ 1  3]
 [ 9 11]]
```

1.4.3 布尔掩码

```
x = np.array([1, -2, 3, -4, 5])
mask = x > 0
print(mask)          # [ True False  True False  True]
print(x[mask])       # [1 3 5]
print(np.where(x > 0, x, 0)) # 正数保留, 否则 0
```

布尔索引返回拷贝；条件组合用 &、|、~ 且必须加括号，例如 `x[(x > 0) & (x < 4)]`。

1.5 视图与拷贝 (重点)

```
a = np.arange(6)
b = a.reshape(2, 3)      # 通常为视图
b[0, 0] = 99             # 99 —— 共享内存!
print(a[0])              # 99 —— 共享内存!

c = a.copy()              # 独立拷贝
c[0] = -1
print(a[0])              # 仍为 99

print(a.base is None)    # 原始数组 base 为 None
print(b.base is a)       # True → b 是 a 的视图
```

经验法则:

操作	是否共享内存
基本切片	视图
reshape (连续内存)	视图 (否则拷贝)
ravel	视图 (尽可能)
flatten	拷贝
花式索引 / 布尔索引	拷贝
copy()	拷贝

1.6 广播机制 (broadcasting)

规则: 从最后一个维度往前比较——(1) 两维相等; (2) 或其中一维为 1 (可“拉伸”); (3) 或该维不存在 (自动补 1)。都不满足则报错。

```
A = np.array([[1, 2, 3], [4, 5, 6]])  # (2,3)
b = np.array([10, 20, 30])           # (3,) → 看作 (1,3)
print(A + b)

[[11 22 33]
 [14 25 36]]
```

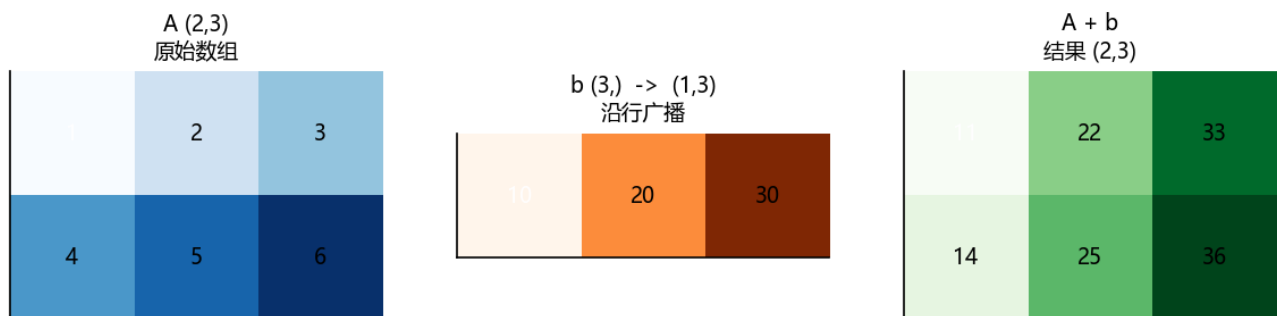


Figure 2: 广播示意图

- $(2,3) + (3,)$ ✓, 结果为 $(2,3)$;
- $(3,) + (4,)$ × (末尾不等且无 1);
- $(4,1) + (1,3)$ ✓, 结果 $(4,3)$;
- $(4,1,6) + (3,6)$ ✓, 结果 $(4,3,6)$ (把 $(3,6)$ 补成 $(1,3,6)$)。

调试技巧：报错时打印两个数组的 shape，用 np.newaxis（即 None）手动补维。

1.7 随机数生成与抽样

1.7.1 新版推荐：Generator

```
rng = np.random.default_rng(42)      # 固定种子，结果可复现
rng.random((2, 3))                   # [0,1) 均匀
rng.integers(1, 10, size=5)           # 整数 [1,10)
rng.uniform(-1, 1, size=4)            # 均匀分布
rng.normal(2, 0.5, size=6)           # 正态  $N(2, 0.5^2)$ 
```

1.7.2 旧接口（仍可用，但不建议用于新代码）

```
np.random.seed(42)
np.random.rand(2, 3)                 # [0,1)
np.random.randint(1, 10)             # 整数
np.random.randn(5)                   # 标准正态
np.random.normal(2, 0.5, 5)
```

选型：新代码优先 default_rng（更快、线程安全、接口清晰）。

常见误区

1. *** 不等于矩阵乘法：** $A * B$ 是逐元素乘法；矩阵乘法用 $A @ B$ 或 np.matmul 。
2. **忘记 zeros/ones/full 要传元组：** $\text{np.zeros}(2, 3)$ 会报错，应为 $\text{np.zeros}((2, 3))$ 。
3. **以为 reshape 一定返回拷贝：**连续内存下是视图；改视图会影响原数组。
4. **误用 len(a)：**一维数组返回长度，二维返回行数；看形状用 $a.\text{shape}$ 。
5. **广播不通用：** $(3,) + (4,)$ 不会自动“截断对齐”。
6. **随机数不设种子：**不设种子结果每次不同，课堂演示/作业请用 $\text{default_rng}(\text{seed})$ 。

思考题

1. $\text{np.arange}(10)[::2]$ 与 $\text{np.arange}(10).\text{reshape}(5,2)[:,0]$ 是否共享内存？用 '.base' 验证。
2. 解释 np.ix_ 与同时传入两个顺序索引的区别。
3. 举一个形状满足广播规则却“符合直觉但结果错误”的例子（提示： $(3,1)$ 与 $(3,)$ ）。

动手练习（详见 lab）

- 创建 3×4 矩阵，用布尔掩码把负数置 0；
- 用 np.arange 生成 0–100 的偶数，转成 10×5 矩阵；
- 复现“对每列做不同缩放”的广播例子；
- 用 $\text{default_rng}(0)$ 生成 100 个标准正态数并统计均值/方差。

延伸阅读

- NumPy 官方绝对入门：https://numpy.org/doc/stable/user/absolute_beginners.html
- NumPy 官方广播机制：<https://numpy.org/doc/stable/user/basics.broadcasting.html>
- SciPy Lecture Notes – NumPy：<https://scipy-lectures.org/intro/numpy/index.html>

02 数组运算：向量、矩阵、逻辑筛选与统计

本节对应原版 1.2 的内容，并增补学习目标、常见误区、思考题与延伸阅读。配套图：images/vector_ops.png。

本节目标

- 区分逐元素运算与线性代数运算 (* vs @ / dot);
- 会用布尔掩码与 np.where 做筛选与替换;
- 掌握 sum/mean/std/min/max 沿 axis 方向统计;
- 会用 sort/argsort 排序与排名;
- 会用 savetxt/loadtxt、save/load、savez_compressed 保存与读取数组。

先修

- 01 数组基础 (创建、索引、广播)。
-

2.1 向量的运算

2.1.1 逐元素操作

```
import numpy as np

v = np.array([2, 3])
w = np.array([3, 1])
print(v + w)      # [5 4]
print(v - w)      # [-1 2]
print(2 * v)       # [4 6]
print(v * w)       # 逐元素乘 [6 3]
print(v / w)       # 逐元素除 [0.66666667 3.]
print(v ** 2)      # 逐元素平方 [4 9]
```

2.1.2 数量积 (点积) 与叉积

```
a = np.array([1, 2])
b = np.array([3, 4])
print(np.dot(a, b))      # 1*3 + 2*4 = 11
print(a @ b)             # 向量点积同样可用 @

c = np.array([1, 2, 3])
d = np.array([4, 5, 6])
print(np.cross(c, d))    # [-3  6 -3]
```

2.1.3 向量的范数

```
x = np.array([3, 4])
np.linalg.norm(x)        # 5.0 (默认 2-范数)
np.linalg.norm(x, ord=1) # 7.0 (1-范数)
```

2.1.4 排序

```
a = np.array([3, 1, 2])
np.sort(a)              # 返回新数组 [1 2 3]
a.sort()                # 原地排序
np.argsort(a)           # 排序后原索引
```

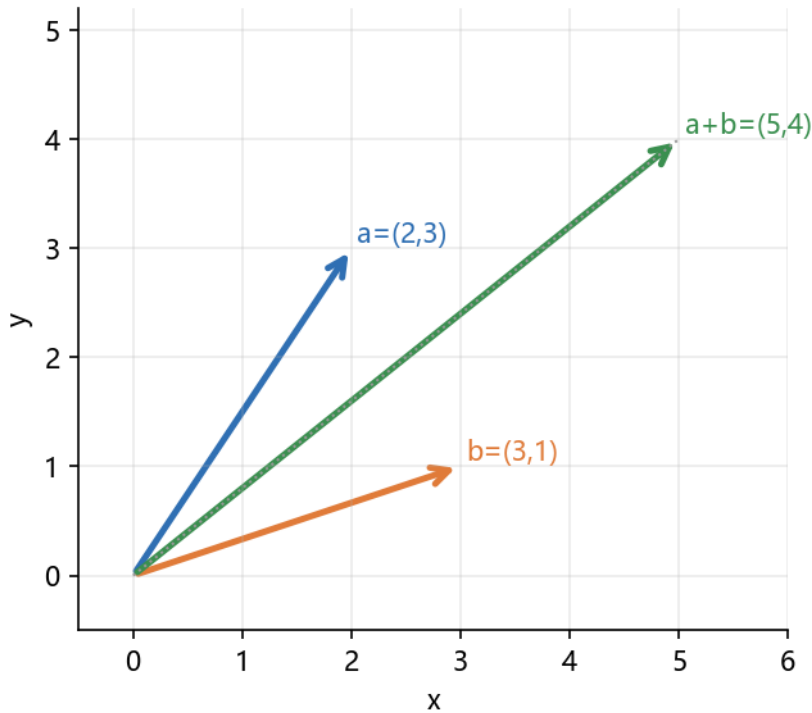


Figure 3: 向量运算示意

2.2 向量/矩阵的逻辑运算——筛选

```
x = np.array([1, -2, 3, -4, 5])
```

布尔掩码

```
print(x[x > 0]) # [1 3 5]
```

```
print(x[(x > 1) & (x < 5)]) # [3]
```

np.where: 条件成立取第一项, 否则取第二项

```
print(np.where(x > 0, x, 0)) # [1 0 3 0 5]
```

二维按行筛选:

```
m = np.array([[1, 9], [6, 2], [7, 8]])
```

```
print(m[(m[:, 0] > 3) & (m[:, 1] < 9)]) # [[6 2] [7 8]]
```

组合条件必须加括号, 且不能用 and / or, 要用 & / | / ~。

2.3 矩阵的运算

2.3.1 算术运算

```
A = np.array([[1, 2], [3, 4]])
```

```
B = np.array([[5, 6], [7, 8]])
```

```
print(A + B) # 矩阵加法 [[ 6 8][10 12]]
```

```
print(A - B) # 矩阵减法
```

```
print(A * B)      # 注意: 逐元素乘 [[ 5 12][21 32]]
print(A @ B)      # ✓ 矩阵乘 [[19 22][43 50]]
print(np.matmul(A, B)) # 同 @
```

为什么 * 不是矩阵乘: NumPy 中 * 是 ufunc 的逐元素运算; 要数学上的矩阵乘法, 用 @ (或 np.dot / np.matmul, 二者在 2D 下等价, 但 @ 语义更明确)。

2.3.2 矩阵上的统计

```
S = np.array([[85, 90, 80],
              [78, 85, 90],
              [90, 95, 85]])

S.sum(axis=0)      # 每列和 [253 270 255] (三门课总分)
S.sum(axis=1)      # 每行和 [255 253 270] (每个学生总分)
S.mean(axis=0)     # 每列均值 [84.33 90. 85. ]
S.max(axis=1)      # 每行最大值
S.std()            # 全体标准差
```

记忆: axis=0 表示 “沿第 0 轴 (行方向) 折叠”, 即求**每列**的统计量; axis=1 求**每行**。

2.3.3 查找 (where / argmax)

```
m = np.array([[3, 7], [9, 2]])
np.where(m > 5)      # (array([0,1]), array([1,0])) 坐标
np.argmax(m)         # 展平后的最大索引 2 → (1,0)
np.argmax(m, axis=1) # 每行最大索引 [1 0]
```

2.4 结果的保存与读取

```
a = np.array([[1, 2], [3, 4]])

## 文本
np.savetxt("result.txt", a, fmt="%.2f", delimiter=",")
b = np.loadtxt("result.txt", delimiter=",")

## 二进制 (推荐: 更快、保留 dtype)
np.save("a.npy", a)
c = np.load("a.npy")

## 多数组压缩保存
np.savez_compressed("data.npz", x=a, y=b)
d = np.load("data.npz")
d["x"], d["y"]
```

常见误区

1. *** 与 @ 混用:** A * B 是逐元素乘; 数学矩阵乘必须 @。
2. **axis 方向记反:** sum(axis=0) 是 “消除第 0 维”, 得到每列结果。
3. **条件用 and/or:** 数组布尔运算必须 &/|, 且每个条件加括号。
4. **反复 np.append:** 每次产生新数组, 大数据循环中极慢; 先估算容量或用列表。
5. **np.dot 语义在多维下容易混淆:** 明确用 @ 更直观。

思考题

1. `np.dot`、`np.matmul`、`@` 在三维数组上行为是否一致？查文档说明。
2. 为什么 `np.where(cond, x, y)` 要求 `x`、`y` 可广播到 `cond` 的形状？
3. 用 `argsort` 给三位学生按总分排名。

动手练习（详见 lab）

- 生成 5×5 随机矩阵，把主对角线元素置 0；
- 对每列做 z-score 标准化 $((x - \text{mean}) / \text{std})$ ，并比较 `axis` 的选择；
- 用 `savetxt/loadtxt` 存取一份成绩表并检查数值一致。

延伸阅读

- NumPy 官方 ndarray 基础：<https://numpy.org/doc/stable/user/quickstart.html>
- NumPy 官方 I/O 说明：<https://numpy.org/doc/stable/reference/routines.io.html>
- SciPy Lecture Notes –数组操作：<https://scipy-lectures.org/intro/numpy/operations.html>

03 线性代数：逆、方程组、分解与 SVD

本节对应原版 1.3 的内容，并增补学习目标、常见误区、思考题与延伸阅读。配套图：`images/cond_sensitivity.png`、`images/svd_compression.png`。

本节目标

- 会求逆矩阵、伪逆、秩、行列式、范数；
- 会用 `np.linalg.solve` 解线性方程组，理解条件数（condition number）对数值稳定的影响；
- 会用 `np.linalg.lstsq` 做最小二乘；
- 掌握特征值分解、SVD、Cholesky、QR 分解的基本用法与适用场景；
- 能说出 SVD 的“压缩/降维”思想（见 05 综合案例）。

先修

- 01 数组基础、02 数组运算；线性代数的基本概念（矩阵、行列式、特征值）。

3.1 基本线性代数运算

3.1.1 逆矩阵与伪逆

```
import numpy as np
```

```
A = np.array([[3, 1], [1, 2]])
A_inv = np.linalg.inv(A)
print(A_inv)
## [[ 0.4 -0.2]
## [-0.2 0.6]]
print(A @ A_inv)      # ≈ 单位矩阵
```

非方阵或奇异矩阵用 `pinv`（伪逆），常用于最小二乘与超定方程组：

```
B = np.array([[1, 2], [2, 4], [3, 6]])  # 秩亏
print(np.linalg.pinv(B).shape)          # (2,3)
```

3.1.2 秩

```
np.linalg.matrix_rank(np.array([[1, 2], [2, 4]])) # 1 (第二行是第一行的 2 倍)
np.linalg.matrix_rank(np.array([[1, 2], [3, 4]])) # 2
```

3.1.3 行列式

```
A = np.array([[3, 1], [1, 2]])
print(np.linalg.det(A)) # 5.0 (近似)
```

行列式为 0 等价于矩阵奇异，此时方程组没有唯一解。

3.1.4 范数

```
v = np.array([3, 4])
np.linalg.norm(v) # 5.0, 2-范数
np.linalg.norm(v, ord=1) # 7.0

M = np.array([[3, 4], [0, 5]])
np.linalg.norm(M) # Frobenius 范数 ≈ 7.0711
np.linalg.norm(M, ord=2) # 谱范数 (最大奇异值)
np.linalg.norm(M, ord=np.inf) # 无穷范数 (行和最大值)
```

3.2 线性方程组的求解

3.2.1 直接求解: np.linalg.solve

```
A = np.array([[3, 1], [1, 2]]) # 系数矩阵
b = np.array([9, 8]) # 常数向量
x = np.linalg.solve(A, b)
print(x) # [2. 3.], 即 x=2, y=3
print(A @ x) # ≈ [9. 8.]
```

适用条件: A 是方阵且非奇异 (存在唯一解)。结构更一般的线性方程组用 lstsq。

3.2.2 最小二乘: np.linalg.lstsq 超定 (方程多于未知数) 或欠定 (方程少于未知数) 时, 求 “误差平方和最小” 的解:

```
X = np.array([[1, 1], [1, 2], [1, 3], [1, 4]])
y = np.array([6, 5, 7, 10])
coef, residuals, rank, sv = np.linalg.lstsq(X, y, rcond=None)
print(coef) # [3.5 1.4] → y ≈ 3.5 + 1.4 x
```

3.2.3 条件数与数值稳定性 条件数 $\text{cond}(A)$ 刻画方程组对抗动的敏感程度: 条件数越大, 解越 “脆弱”。

```
A = np.array([[1, 2, 3],
               [2, 4.0001, 6],
               [1, 0.9999, 2]])
b = np.array([1, 2, 3])
print(np.linalg.cond(A)) # 这个矩阵近似奇异, cond 很大
x0 = np.linalg.solve(A, b)
```

结论: 即使 b 的扰动非常小 ($1e-4$ 量级), 解的变化也可能很大; 工程中应尽量避免病态方程组, 或换用更稳定的分解方法。

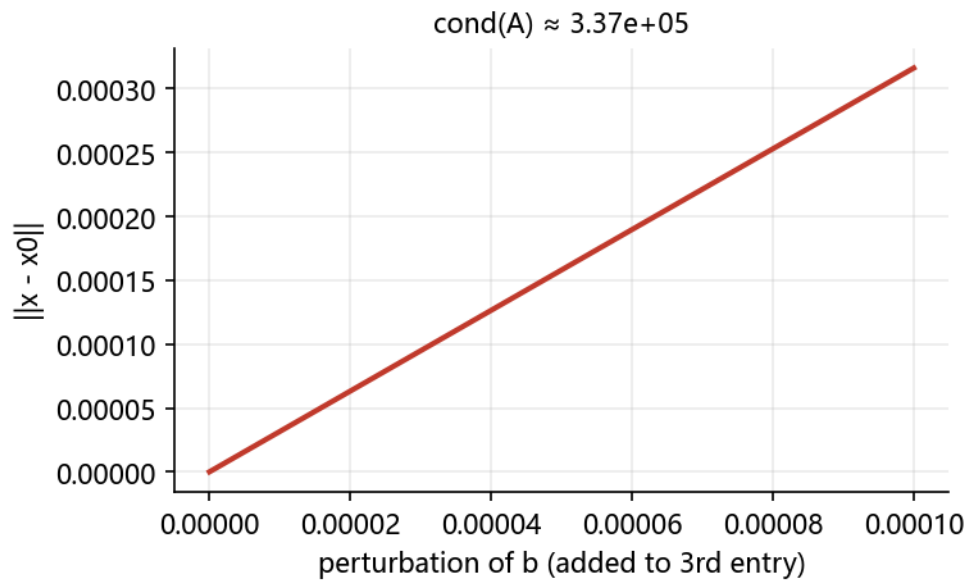


Figure 4: 条件数敏感性

3.3 矩阵分解

3.3.1 特征值分解 (eig) 只适用于**方阵** (且通常要求可对角化):

```
A = np.array([[2, 1], [1, 2]])
w, v = np.linalg.eig(A)
print(w)           # [3. 1.] 特征值
print(v)           # 列向量为对应特征向量
print(A @ v[:, 0] == w[0] * v[:, 0]) # 验证  $Av = \lambda v$ 
```

3.3.2 奇异值分解 (SVD) 对**任意矩阵** (包括非方阵) 适用:

```
M = np.array([[1, 2], [3, 4], [5, 6]], dtype=float)
U, S, Vt = np.linalg.svd(M, full_matrices=False)
print(U.shape, S.shape, Vt.shape) # (3,2) (2,) (2,2)
print(S)                         # 奇异值 (降序)
print(U @ np.diag(S) @ Vt)       # 重构  $\approx M$ 
```

为什么重要: 奇异值从大到小排列, 只保留前 k 个奇异值即可得到“最优低秩近似”, 这是数据压缩、降维 (PCA)、图像压缩的基础。见 [05 综合案例](#)。

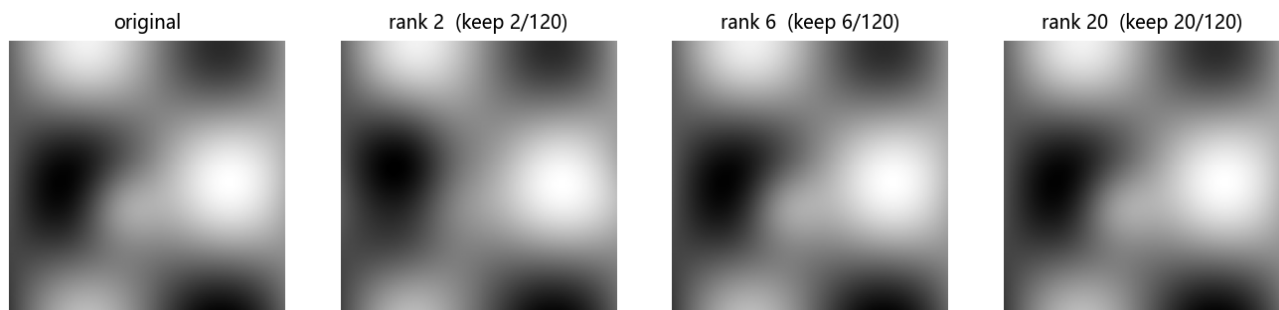


Figure 5: SVD 压缩示例

3.3.3 Cholesky 分解 要求矩阵对称正定:

```
A = np.array([[4., 2.], [2., 3.]])
L = np.linalg.cholesky(A)
print(L)          # [[2. 0.] [1. 1.4142]]
print(L @ L.T)    # ≈ A
```

3.3.4 QR 分解

```
A = np.array([[12., -51.], [6., 167.], [-4., 24.]])
Q, R = np.linalg.qr(A)
print(Q.shape, R.shape)  # (3,2) (2,2)
print(np.allclose(Q @ R, A))  # True
```

QR 在最小二乘、特征值算法稳定实现中广泛使用。

常见误区

1. 对非方阵用 `inv`: 应使用 `pinv` 或 `lstsq`。
2. 认为 `eig` 对任意矩阵都可用: 特征值分解只对方阵; 非方阵用 SVD。
3. 忽略条件数: `solve` 计算成功 $\neq$ 解可靠, 先看 `cond`。
4. SVD 返回的 `S` 不是对角矩阵: 需要 `np.diag(S)` 构造。
5. `full_matrices=False` 与非默认的区别: 不关心零奇异值/补全正交基时, 用 `full_matrices=False` 更省内存。

思考题

1. 为什么 `cond(A)` 很大时, `np.linalg.solve(A, b)` 的结果可能仍 “看起来正常”?
2. SVD 的 `U`、`Vt` 与 `eig` 的特征向量有什么联系 (提示: $A^T A$)?
3. 若只保留前 1 个奇异值做近似, 误差如何估计?

动手练习 (详见 lab)

- 解一个 4×4 线性方程组 3 组 (含病态) 并比较解;
- 对 10×5 矩阵做 SVD, 比较 rank 1/3/5 重构误差;
- 用 Cholesky 验证正定矩阵的分解唯一性。

延伸阅读

- NumPy 官方 linalg 参考: <https://numpy.org/doc/stable/reference/routines.linalg.html>
- NumPy 官方广播与 linalg 教程: <https://numpy.org/doc/stable/reference/routines.linalg.html#module-numpy.linalg>
- SciPy Lecture Notes –线性代数: <https://scipy-lectures.org/intro/scipy.html#linear-algebra>

04 多项式与拟合: poly1d、求根与最小二乘拟合

本节对应原版 1.4 的内容, 并增补学习目标、常见误区、思考题与延伸阅读。配套图: `images/polyfit.png`。

本节目标

- 会用 `np.poly1d` 构造、求值、加减乘除多项式;
- 会求导 (`deriv`)、积分 (`integ`)、求根 (`np.roots`);
- 会用 `np.polyfit` / `np.polyval` 做最小二乘拟合, 并理解 “过拟合” 现象;

- 知道新版 `np.polynomial.Polynomial` 的方向（便于迁移）。

先修

- 01 数组基础、02 数组运算；了解“多项式系数按最高次到常数排列”。
-

4.1 多项式的构造与运算

```
import numpy as np
```

```
## 2x^2 + 3x + 1
p = np.poly1d([2, 3, 1])
print(p)          # 2 x^2 + 3 x + 1
print(p(5))       # 66

2
2 x + 3 x + 1
66
```

4.1.1 四则运算

```
p = np.poly1d([2, 3, 1])      # 2x^2+3x+1
q = np.poly1d([1, -1])       # x - 1

print(p + q)                  # 加法
print(p - q)                  # 减法
print(p * q)                  # 乘法（系数卷积）
print(p / q)                  # 除法 →（商，余数）元组

quotient, remainder = p / q   # poly1d 直接除返回（商，余数）元组
print(quotient)               # 商
print(remainder)              # 余数

## 也可用 numpy.polydiv（传入系数）
q_, r_ = np.polydiv(p.coeffs, q.coeffs)
```

4.1.2 求导与积分

```
p = np.poly1d([2, 3, 1])
print(p.deriv())              # 4x + 3
print(p.deriv(2))             # 二阶导
print(p.integ())              # 0.6667x^3 + 1.5x^2 + x（常数默认为 0）
print(p.integ(m=2))           # 二重积分
```

4.1.3 系数与求根

```
p = np.poly1d([1, -3, 2])     # x^2 - 3x + 2
print(p.coeffs)                # [ 1. -3.  2.]
print(p.c)                     # 按降幂的系数
np.roots(p)                    # [2. 1.]
```

`np.roots` 也支持直接传系数数组：`np.roots([1, -3, 2])`。

4.2 最小二乘拟合: polyfit

```
x = np.array([0, 1, 2, 3, 4], dtype=float)
y = np.array([1, 2, 9, 28, 65], dtype=float) # 恰好是  $x^3 + 1$ 
```

```
coef = np.polyfit(x, y, 3) # 3 次多项式最小二乘
print(coef) # [ 1.  0. -0.  1.]
p = np.poly1d(coef)
print(p) # 1 x^3 + 0 x^2 + 0 x + 1
print(np.polyval(coef, 2.5)) # 16.625
```

含噪声数据的拟合与过拟合:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
np.random.seed(7)
def f(x): return 2 * x ** 2 - 3 * x + 1
x = np.linspace(-2, 3, 40)
y = f(x) + np.random.normal(0, 2, x.size) # 加噪声
```

```
for deg in (1, 3, 5):
    coef = np.polyfit(x, y, deg)
    print(deg, ":", coef)
```

绘图

```
xs = np.linspace(-2, 3, 200)
plt.scatter(x, y, s=16, label="data")
for deg, c, ls in [(1, "grey", "--"), (3, "orange", "-."), (5, "green", "-")]:
    plt.plot(xs, np.polyval(np.polyfit(x, y, deg), xs), c=c, ls=ls, label=f"deg {deg}")
plt.legend(); plt.savefig("poly_fit_demo.png")
```

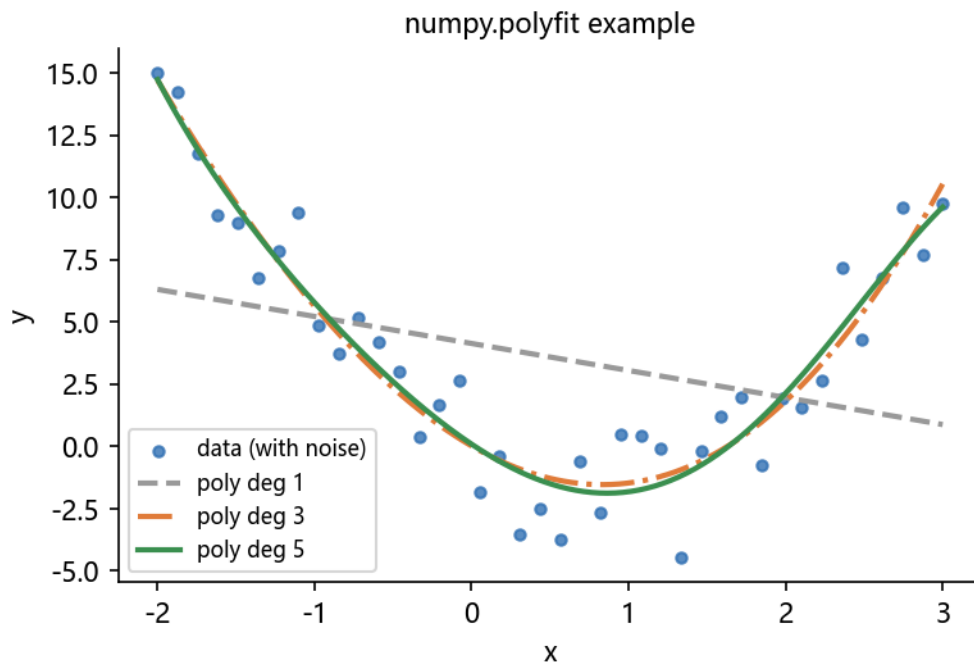


Figure 6: polyfit 示例

观察：次数越高，拟合曲线越可能贴近噪声（过拟合）；次数过低则拟合不足。选择次数可用交叉验证或残差分析，而不是盲目升高次数。

4.3 新版多项式接口（了解）

NumPy 1.x 之后的 `np.polynomial` 提供更现代、更稳定的 `Polynomial` / `Chebyshev` 等类：

```
from numpy.polynomial import Polynomial
p = Polynomial([1, 3, 2]) # 注意：系数从常数项开始！
print(p(2))              # 1 + 3*2 + 2*2^2 = 15
print(p.roots())
```

`np.poly1d` 是旧接口（系数降幂、基于 `np.poly`），大量教程/教材仍在使用；新代码可优先考虑 `np.polynomial`。本讲义两者都给出，以兼容常见场景。

常见误区

1. **系数顺序搞反：**`poly1d([a, b, c])` 表示 $ax^2 + bx + c$ ；`np.polynomial.Polynomial` 则是常数项在前。
2. **`polyfit` 次数过高：**数据少时高次多项式会震荡/过拟合。
3. **忘记噪声与残差：**拟合不是“解释唯一真理”，要报告残差与预测区间。
4. **把 `polyfit` 当精确解：**它是数值最小二乘，数据含噪声时系数有误差。

思考题

1. n 个数据点做 $n - 1$ 次多项式拟合会出现什么现象？为什么？
2. `polyfit` 与 `lstsq` 的关系是什么？
3. 如何判断“次数挑对了”？

动手练习（详见 lab）

- 对 $y = 0.5x^3 - 2x + 1 + \varepsilon$ 的 30 个噪声点，分别拟合 1/3/5 次，比较残差与测试误差；
- 用 `poly1d` 求 $x^4 - 5x^2 + 4$ 的根，并验证代入为 0。

延伸阅读

- NumPy 官方 `polynomial` 文档：<https://numpy.org/doc/stable/reference/routines.polynomials.html>
- NumPy 官方 `polyfit` 说明：<https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html>
- SciPy Lecture Notes - 最小二乘：<https://scipy-lectures.org/intro/scipy.html#least-squares-approximation>

05 综合案例：SVD 图像压缩与实验数据拟合

本章新增综合案例，用于把 01-04 的知识串起来：**形状/广播 + 线性代数 + 拟合 + 可视化**。所有代码均可在 NumPy ≥ 1.24 环境直接运行；配图由 `build/make_chap1_figures.py` 生成。

案例 1：用 SVD 压缩一张“图像”

背景 一张 $m \times n$ 的灰度图可看作矩阵。SVD 给出

$$A = U\Sigma V^T, \quad U \in \mathbb{R}^{m \times r}, \Sigma \in \mathbb{R}^{r \times r}, V \in \mathbb{R}^{n \times r}$$

只保留前 k 个奇异值，得到低秩近似

$$A_k = U_{:,1:k} \Sigma_{1:k,1:k} V_{1:k,:}^T$$

存储量从 mn 减少到 $k(m + n + 1)$, 当 $k \ll \min(m, n)$ 时压缩明显。

代码：生成合成图像并做低秩近似

```
import numpy as np
import matplotlib.pyplot as plt

## 1) 生成一张“灰度图” (120x120, 含图案与明暗渐变)
x = np.linspace(0, 1, 120); y = np.linspace(0, 1, 120)
X, Y = np.meshgrid(x, y)
img = (np.sin(6 * X) * np.cos(6 * Y)
        + np.exp(-((X - 0.4) ** 2 + (Y - 0.6) ** 2) * 40)
        + 0.6 * X - 0.4 * Y)
img = (img - img.min()) / (img.max() - img.min())

## 2) SVD
U, S, Vt = np.linalg.svd(img, full_matrices=False)

## 3) 不同秩的重建与误差
for k in (2, 6, 20):
    A_k = U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]
    mse = np.mean((img - A_k) ** 2)
    params = k * (img.shape[0] + 1 + img.shape[1])
    print(f"rank={k:2d} MSE={mse:.5f} 存储量={params} / 原图={img.size} "
          f"({100*params/img.size:.1f}%)")

rank= 2 MSE=0.00087 存储量 =482 / 原图 =14400 (3.3%)
rank= 6 MSE=0.00000 存储量 =1446 / 原图 =14400 (10.0%)
rank=20 MSE=0.00000 存储量 =4820 / 原图 =14400 (33.5%)
```

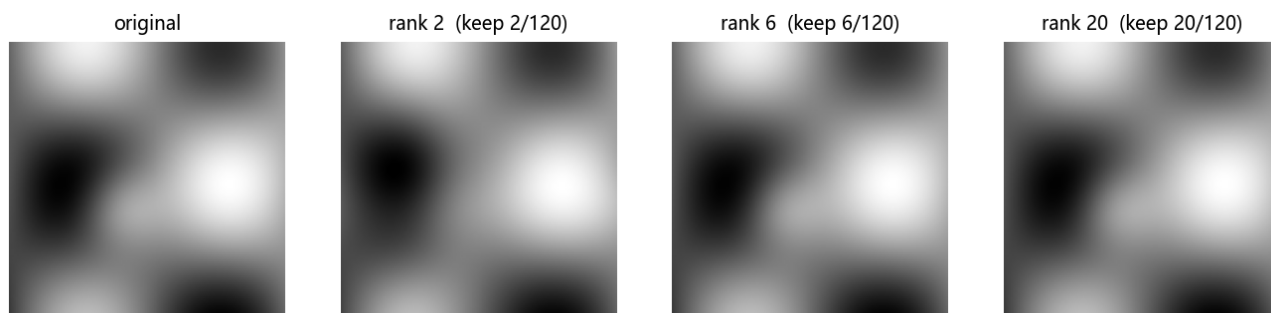


Figure 7: SVD 图像压缩

分析

- rank=2 时仅用 3.3% 的存储量， $MSE \approx 0.0009$ （肉眼几乎无差别）；
- rank=6 之后误差已低于打印精度；
- 说明自然图像的能量高度集中在前几个奇异值——这正是图像压缩、PCA、数据降维的共同原理。

拓展任务

1. 用 `np.linalg.norm(img - A_k, 'fro')` 计算相对误差;
 2. 对一张真实图片 (PIL 读取灰度化) 重复实验, 观察 “压缩率 vs 视觉质量”;
 3. 把 `img` 换成随机噪声图, 观察奇异值衰减是否还那么快? 为什么?
-

案例 2: 实验数据的最小二乘拟合

背景 物理实验得到一组 (x_i, y_i) , 理论公式是二次函数, 但测量有噪声。用 `np.polyfit` 求 $y = ax^2 + bx + c$ 的最佳系数, 并评估拟合质量。

代码

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(7)
def theory(x): return 2 * x ** 2 - 3 * x + 1

x = np.linspace(-2, 3, 40)
y = theory(x) + np.random.normal(0, 2, x.size) # 加噪声

coef2 = np.polyfit(x, y, 2) # 二次拟合
coef5 = np.polyfit(x, y, 5) # 五次拟合 (对比)
print(" 二次系数 (a,b,c):", np.round(coef2, 3))
print(" 五次系数:", np.round(coef5, 3))

## 残差 (RMSE)
rmse2 = np.sqrt(np.mean((np.polyval(coef2, x) - y) ** 2))
rmse5 = np.sqrt(np.mean((np.polyval(coef5, x) - y) ** 2))
print(f"RMSE(deg2)={rmse2:.3f} RMSE(deg5)={rmse5:.3f}")

二次系数 (a,b,c): [ 2.123 -3.231  0.616]
五次系数: [-0.010  0.210 -0.278  1.206 -2.469  1.162]
RMSE(deg2)=2.116 RMSE(deg5)=2.026
```

分析

- 二次拟合系数接近真实值 $(2, -3, 1)$, 说明模型设定正确时最小二乘能恢复参数;
- 五次拟合在训练数据上的 RMSE 略低 (2.026 vs 2.116), 说明 “次数越高越能贴合训练数据” ——但这是 **过拟合的征兆**;
- 把前 28 个点作训练、后 12 个点作**独立测试**时: deg2 测试 RMSE $\approx$ 3.11, 而 deg5 测试 RMSE $\approx$ 139.1! 过拟合在测试集上暴露无遗。因此选次数必须看独立测试集/交叉验证, 而不是训练误差。

拓展任务

1. 划分训练/测试集 (如 70/30), 比较 deg=2 与 deg=5 在测试集上的 RMSE (本案例中 deg2 $\approx$ 3.11, deg5 $\approx$ 139.1);
 2. 用 `np.linalg.lstsq` 重写拟合, 体会与 `polyfit` 的关系;
 3. 打印 95% 置信区间 (提示: 用 `np.polyfit` 的 `cov` 参数或 `scipy.stats`)。
-

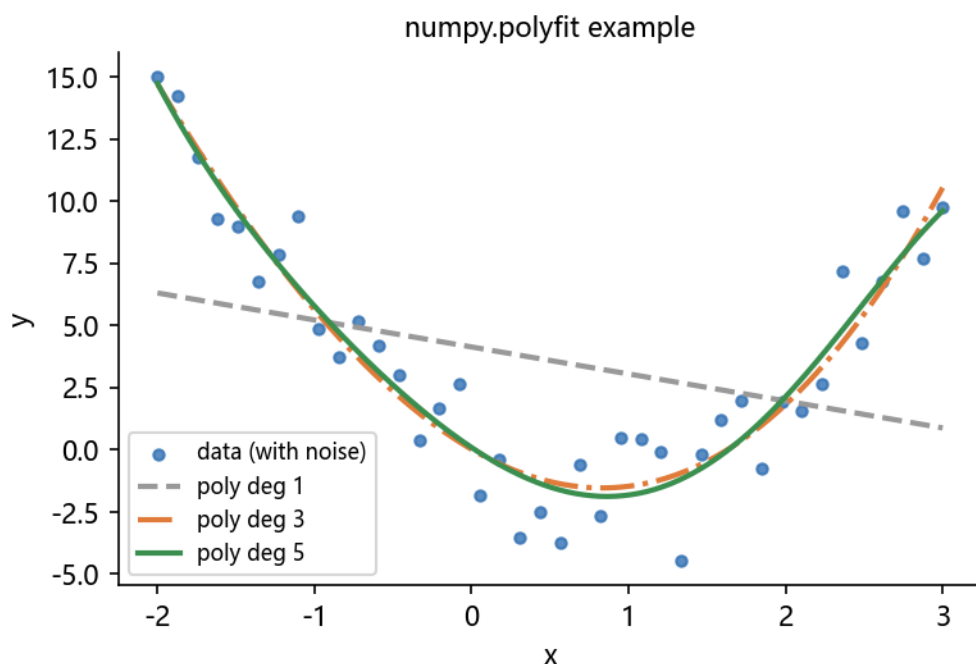


Figure 8: polyfit 示例

本章综合练习（上机任务）

1. 生成 50×50 的“数字 + 渐变”图案，用 $\text{rank} = 1, 3, 5, 10$ 重构并绘制 2×2 对比图；
2. 构造 $y = 3e^{-x} + 2 + \varepsilon$ 的噪声数据，尝试多项式拟合（哪种次数更合理？），并给出误差曲线；
3. 把案例 1 与案例 2 整理成一个 .ipynb，写一份 200 字以内的结论。

延伸阅读

- NumPy 官方 SVD: <https://numpy.org/doc/stable/reference/generated/numpy.linalg.svd.html>
- SciPy 相关低秩近似(scipy.sparse.linalg): <https://docs.scipy.org/doc/scipy/reference/sparse.linalg.html>
- scikit-learn PCA 官方示例(下一章会用到): https://scikit-learn.org/stable/auto_examples/index.html#decomposition

06 常见误区与技巧

本章为新增“查漏补缺”页：把学生最容易踩的坑集中列出，并给出正确的写法和性能技巧。

1. 高频误区速查表

#	误区 / 错误写法	正确做法	原因
1	$A * B$ 当矩阵乘法	$A @ B$ 或 <code>np.matmul(A, B)</code>	$*$ 是逐元素乘 (ufunc)
2	<code>np.zeros(2, 3)</code>	<code>np.zeros((2, 3))</code>	形状要传元组，两个括号
3	以为 reshape 是拷贝	需要独立数据用 <code>arr.reshape(...).copy()</code>	连续内存的 reshape 是视图
4	<code>x[x > 0 and x < 5]</code>	<code>x[(x > 0) & (x < 5)]</code>	数组布尔运算用 $\&$ ，且加括号
5	<code>len(a)</code> 猜数组维数	<code>a.shape</code> 、 <code>a.ndim</code>	<code>len</code> 只给第一维长度
6	对非方阵 <code>np.linalg.inv</code>	<code>np.linalg.pinv</code> / <code>lstsq</code>	逆只定义于方阵

#	误区 / 错误写法	正确做法	原因
7	np.linalg.svd(M) 直接 U@S@Vt	U @ np.diag(S) @ Vt	S 是奇异值向量, 不是对角阵
8	认为 eig 适用于任意矩阵	方阵用 eig, 任意矩阵用 svd	eig 数学条件限制
9	np.random.rand 不设种子	rng = np.random.default_rng(42)	结果不可复现
10	循环里 np.append	预分配数组或列表收集	每次 append 都会新建数组
11	混淆 axis=0/1	记 “消除哪一维就写哪一维”	sum(axis=0) 消行得列统计
12	用 astype 想 “改原数组”	arr.astype 返回新数组; 原地改 dtype 不可直接	类型转换是拷贝

2. 原理级难点: 视图 vs 拷贝

```
a = np.arange(6)
b = a.reshape(2, 3)      # 视图
b[0, 0] = 99
print(a[0])              # 99

c = a.copy()
c[0] = 0                 # 不影响 a
```

判断是否共享内存:

```
b.base is a              # True → 视图
np.may_share_memory(a, b) # 更稳的检查
```

为什么重要: 视图省内存、速度快; 但并发修改或函数返回到别人手里时容易 “隐形改数据”。对外返回数据前, 若不想共享, 显式 .copy()。

3. 广播规则速记

1. 从右往左对齐;
2. 相同或一方为 1 → 兼容;
3. 全部兼容 → 结果为各维最大; 否则 ValueError。

```
A = np.zeros((4, 1, 6))
B = np.zeros((3, 6))
try:
    print((A + B).shape)      # (4, 3, 6), B 自动补成 (1,3,6)
except ValueError as e:
    print(" 广播失败: ", e)
```

注意: (3,) + (4,) 不兼容; (3,1) + (3,) 也不兼容 (末尾 1 vs 3)。想 “列向量与行向量做外积” 要用 (3,1) + (1,3)。

4. 性能技巧

- **向量化优先:** 能一次 `a[np.newaxis, :] + b[:, np.newaxis]` 就不要双重循环;
- **out= 原地计算:** `np.add(a, b, out=a)` 避免额外分配;
- **in-place 算术:** `a += b` (注意类型安全);

- **einsum 表达张量收缩**: `np.einsum('ik,kj->ij', A, B)` 既是矩阵乘, 也能表达更复杂收缩且减少中间数组;
- **`np.lib.stride_tricks.sliding_window_view`** 做滑窗 (移动平均/卷积) 时避免显式复制:


```
x = np.arange(10)
w = np.lib.stride_tricks.sliding_window_view(x, 3)
print(w.mean(axis=1))    # 窗口大小为 3 的移动平均
```
- **大数组用 `np.savez_compressed` 存储**; 文本 `savetxt` 慢且丢精度。

5. 调试建议

1. 出错先打印 `shape / dtype`, 再用 `np.allclose` 比较;
2. 用 `np.set_printoptions(precision=4, suppress=True)` 让输出可读;
3. 复杂布尔条件拆成中间变量 (`mask1 = ...; mask2 = ...`);
4. 随机/拟合问题**固定种子**, 便于复现与讨论;
5. 用 `np.isnan / np.isinf` 检查异常值。

6. 本章小结 (自测)

- ☐ 能解释 `dtype` 与 `itemsize`;
- ☐ 能说出 `view` vs `copy` 的 4 种情形;
- ☐ 能判断 `(2,3)+(3,)`、`(4,1)+(1,3)` 的可广播性;
- ☐ 能区分 `*`、`@`、`np.dot`;
- ☐ 能用 `solve / lstsq` 解不同形态方程组;
- ☐ 能用 `svd / polyfit` 完成 05 综合案例。

延伸阅读

- NumPy 官方 “NumPy for MATLAB users”: <https://numpy.org/doc/stable/user/numpy-for-matlab-users.html>
- NumPy 性能建议: <https://numpy.org/doc/stable/user/performance.html>